



PYTHON · LEVEL UP TO PYTHONIC CODE

파이썬 완전정복

2권 · 심화편

진짜 파이썬다운 코드를 쓰는 법

1권을 댄 당신을 위해, 왕초보 티를 벗고

프로처럼 쓰는 심화 개념을 비유로 쉽게. 모든 예제는 실제로 실행해 검증했습니다.

비유로 이해하는 심화 개념

실행 결과까지 확인

12개 장 · 실전 프로젝트

지은이 · Joon

목차

CONTENTS · 2 권 심 화 편

- 01 객체지향 프로그래밍(OOP) 입문 클래스와 객체(붕어빵 비유), `__init__`, `self`
- 02 OOP 더 깊이 상속·오버라이딩·`super()`·캡슐화·특수메서드·`property`
- 03 컴프리헨션과 표현식 리스트·딕셔너리·셋 컴프리헨션, 조건부 표현식
- 04 이터레이터와 제너레이터 `iter/next`, `yield`, 메모리 효율, 무한 시퀀스
- 05 함수를 자유자재로 일급 함수·람다·`map/filter`·sorted key·클로저
- 06 데코레이터 함수를 감싸는 함수(선물 포장), `@` 문법, `wraps`
- 07 정규표현식(re) 패턴 매칭, `search/findall/sub`, 실전 추출
- 08 유용한 표준 라이브러리 `collections`·`itertools`·`functools`·`pathlib`·`json`
- 09 타입 힌트와 좋은 코드 타입 어노테이션·`typing`·`docstring`·PEP8
- 10 파이썬답게: 자주 쓰는 패턴 언패킹·`enumerate/zip`·`with`·`*args/**kwargs`
- 11 동시성 맛보기 `threading` vs `multiprocessing`·`concurrent.futures`
- 12 실전 프로젝트 도서 관리 시스템 · 로그 분석기

객체지향 프로그래밍(OOP) 입문

1권에서 우리는 변수·함수·리스트로 프로그램을 만들었어요. 그런데 프로그램이 커지면 '데이터'와 '그 데이터를 다루는 기능'이 여기저기 흩어져 관리하기 힘들어져요. 이것을 하나로 묶어 깔끔하게 설계하는 방법이 바로 **객체지향 프로그래밍**이에요. 겁먹지 마세요. 붕어빵 하나로 다 이해할 수 있어요.

이 장에서 배울 것

- 클래스와 객체가 무엇인지 (붕어빵 틀 vs 붕어빵)
- `__init__` 과 `self` 의 정체
- 속성(데이터)과 메서드(기능) 만들기
- 인스턴스를 여러 개 찍어내기

1.1 클래스와 객체 — 붕어빵 틀과 붕어빵

클래스(class)는 설계도예요. **객체(object)**는 그 설계도로 찍어낸 실제 물건이고요. 가장 좋은 비유가 붕어빵이에요.

- **붕어빵 틀** = 클래스. 모양과 만드는 법이 정해진 설계도.
- **구워낸 붕어빵** = 객체(인스턴스). 틀로 찍어낸 실물.
- 틀 하나로 붕어빵을 여러 개 찍을 수 있어요. 각 붕어빵은 팔·슈크림처럼 속이 다를 수 있죠.

즉 클래스를 한 번 정의해 두면, 그걸로 객체를 원하는 만큼 만들 수 있어요. 객체마다 자기만의 데이터를 가질 수 있고요.



용어 정리

- **클래스**: 설계도 (붕어빵 틀)
- **객체 / 인스턴스**: 설계도로 만든 실물 (구워낸 붕어빵)
- **속성(attribute)**: 객체가 가진 데이터 (붕어빵의 속·크기)
- **메서드(method)**: 객체가 할 수 있는 동작 (함수인데 클래스 안에 있는 것)

1.2 첫 클래스 만들기

클래스는 `class 이름:` 으로 만들어요. 이름은 관례로 **대문자로 시작**해요(예: `Dog`, `BankAccount`). 강아지를 표현하는 클래스를 만들어 볼게요.

파이썬 코드

```
class Dog:
    def __init__(self, name, age):
        self.name = name    # 속성: 이름
        self.age = age      # 속성: 나이

    def bark(self):         # 메서드: 짖기
        print(f"{self.name}: 멍멍!")

d = Dog("바둑이", 3)    # 객체 만들기(인스턴스화)
print(d.name, d.age)
d.bark()
```

실행 결과

```
바둑이 3
바둑이: 멍멍!
```

`d = Dog("바둑이", 3)` 이 줄이 바로 붕어빵을 굽는 순간이에요. `Dog` 라는 틀로 `d` 라는 강아지 한 마리를 만들었죠. 만든 객체의 속성은 `d.name` 처럼 점(.)으로 꺼내 쓰고, 동작은 `d.bark()` 처럼 점으로 불러요.

1.3 __init__ 과 self 파헤치기

두 낱선 녀석이 있었죠. 하나씩 정확히 짚어 볼게요.

▶ __init__ — 객체가 태어날 때 자동 실행

`__init__` (던더 이닛)은 생성자라고 불러요. 객체를 만드는 순간 **자동으로 딱 한 번 실행**되는 특별한 메서드예요. 여기서 객체의 초기 속성을 설정해요. 붕어빵으로 치면 '틀에 반죽과 팔을 붓는 순간'이죠. `Dog("바둑이", 3)` 라고 부르면 `__init__` 의 `name` 에 "바둑이", `age` 에 3이 전달돼요.

▶ self — 자기 자신을 가리키는 말

`self` 는 지금 이 객체 자신을 가리켜요. 붕어빵 여러 개를 굽는데, "이 붕어빵의 속"을 정할 때 어느 붕어빵인지 가리켜야 하잖아요? 그게 `self` 예요. `self.name = name` 은 '이 객체의 `name` 속성에 값을 저장하라'는 뜻이에요.



메서드의 첫 번째 매개변수는 항상 `self` 예요(관례). 그런데 `d.bark()` 라고 부를 때 우리는 `self` 를 안 넣죠? 파이썬이 `d` 를 `self` 자리에 **자동으로** 넣어 주기 때문이에요. 그래서 `self` 는 '내가 부른 그 객체'가 돼요.

1.4 객체를 여러 개 찍어내기

클래스의 진짜 힘은 여기 있어요. 틀 하나로 서로 다른 객체를 여러 개 만드는 거죠. 각 객체는 자기만의 속성값을 가져요.

파이썬 코드

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(f"{self.name}(이)가 짖어요: 멍멍!")

a = Dog("바둑이")
b = Dog("초코")
a.bark()
b.bark()
print(a.name, b.name)
```

실행 결과

```
바둑이(이)가 짖어요: 멍멍!
초코(이)가 짖어요: 멍멍!
바둑이 초코
```

`a` 와 `b` 는 같은 `Dog` 틀로 만들었지만 완전히 다른 강아지예요. `a.name` 은 "바둑이", `b.name` 은 "초코"로 각자 자기 데이터를 갖고 있죠. 이게 붕어빵 틀 하나로 팔·슈크림 붕어빵을 따로 굽는 것과 같아요.

1.5 상태를 바꾸는 메서드

객체는 자기 속성을 바꾸는 메서드도 가질 수 있어요. 카운터(계수기)를 만들어 볼게요.

파이썬 코드

```
class Counter:
    def __init__(self):
        self.count = 0

    def increase(self):
        self.count += 1

    def show(self):
        print(f"현재 값: {self.count}")

c = Counter()
c.increase()
c.increase()
c.increase()
c.show()
```

실행 결과

현재 값: 3

`increase()` 를 부를 때마다 `self.count` 가 1씩 올라가요. 객체가 자기 상태를 기억한다는 게 함수만 쓸 때와 다른 점이에요. 이 '기억하는 데이터 + 그걸 다루는 기능'의 묶음이 객체지향의 핵심 아이디어예요.

1.6 조금 더 실전다운 예 — 은행 계좌

이번엔 입금·출금이 되는 계좌를 만들어 볼게요. 잔액이라는 데이터와 그걸 다루는 기능이 한 클래스 안에 자연스럽게 묶여요.

```

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"{amount}원 입금 \u2192 잔액 {self.balance}원")

    def withdraw(self, amount):
        if amount > self.balance:
            print("잔액이 부족합니다.")
        else:
            self.balance -= amount
            print(f"{amount}원 출금 \u2192 잔액 {self.balance}원")

acc = BankAccount("지우", 10000)
acc.deposit(5000)
acc.withdraw(3000)
acc.withdraw(99999)

```

실행 결과

```

5000원 입금 → 잔액 15000원
3000원 출금 → 잔액 12000원
잔액이 부족합니다.

```

`balance=0` 처럼 기본값도 줄 수 있어요(1권 함수에서 배웠죠). 출금할 때 잔액을 확인하는 규칙까지 메서드 안에 담겨 있다는 게 포인트예요. 데이터(잔액)와 규칙(출금 검사)이 한 몸이라 안전하고 관리하기 쉬워요.

**흔한 실수: self 를 빠뜨려요**

메서드 안에서 속성에 접근할 땐 반드시 `self.` 를 붙여야 해요. `balance += amount` 라고만 쓰면 '그런 변수 없다'는 오류가 나요. `self.balance` 라고 써야 '이 객체의 잔액'을 가리켜요.

연습문제

1. 반지름을 받아 넓이를 돌려주는 `Circle` 클래스를 만들어 보세요. 넓이 = $3.14 * \text{반지름} * \text{반지름}$.
`Circle(5).area()` 는 `78.5`.
2. `name`, `hp` (체력)를 가진 `Monster` 클래스를 만들고, `attack(damage)` 메서드로 체력을 깎아 보세요.
3. 위 `Counter` 에 `reset()` 메서드를 추가해 `count` 를 0으로 되돌려 보세요.

해답 · 힌트

① 예시 답안

```
class Circle:
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius * self.radius

print(Circle(5).area())    # 78.5
```

② `attack` 안에서 `self.hp -= damage`. ③ `def reset(self): self.count = 0`.

OOP 더 깊이

클래스의 기본을 잡았으니, 이제 객체지향을 '진짜 무기'로 만들어 주는 개념들을 배워요. 부모의 성질을 물려받는 **상속**, 물려받되 내 방식으로 바꾸는 **오버라이딩**, 그리고 `len()`, `print()` 같은 파이썬 문법이 내 객체에도 통하게 만드는 **특수 메서드**까지. 여기까지 오면 남의 코드도 훨씬 잘 읽혀요.

이 장에서 배울 것

- 상속으로 코드 재사용하기 (부모/자식 클래스)
- 메서드 오버라이딩과 `super()`
- 캡슐화 — `_` 와 `__` 로 감추기
- 특수 메서드(`__str__`, `__len__` 등)와 `@property`

2.1 상속 — 물려받아 재사용하기

상속(inheritance)은 어떤 클래스가 다른 클래스의 속성·메서드를 그대로 물려받는 거예요. 비유하자면 '동물'이라는 큰 틀이 있고, '고양이'는 동물의 성질을 물려받으면서 자기만의 특징(야옹)을 더하는 거죠. 물려주는 쪽을 **부모(상위) 클래스**, 물려받는 쪽을 **자식(하위) 클래스**라고 해요.

파이썬 코드

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name}(이)가 소리를 냅니다.")

class Cat(Animal):          # Animal 을 물려받음
    def meow(self):
        print(f"{self.name}: 야옹~")

c = Cat("나비")
c.speak()    # 부모에게 물려받은 메서드
c.meow()     # 자식만의 메서드
```

실행 결과

```
나비(이)가 소리를 냅니다.
나비: 야옹~
```

`class Cat(Animal):` 처럼 괄호 안에 부모를 적으면 상속이 돼요. `Cat` 은 `speak()` 를 직접 만들지 않았는데도 쓸 수 있죠? `Animal` 에게 물려받았으니까요. 공통 기능은 부모에 한 번만 만들어 두고 여러 자식이 나눠 쓰니, 코드가 확 줄어요.

2.2 오버라이딩 — 물려받되 내 방식으로

자식이 부모의 메서드를 같은 이름으로 다시 정의하면, 자식 걸로 **덮어써져요**. 이것 **오버라이딩(overriding)**이라고 해요. 동물은 그냥 '소리를 낸다'지만, 개는 '멍멍', 고양이는 '야옹'으로 각자 다르게 바꾸는 거죠.

파이썬 코드

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name}: (소리)")

class Dog(Animal):
    def speak(self):          # 오버라이딩
        print(f"{self.name}: 멍멍!")

class Cat(Animal):
    def speak(self):          # 오버라이딩
        print(f"{self.name}: 야옹~")

for a in [Dog("바둑이"), Cat("나비")]:
    a.speak()
```

실행 결과

```
바둑이: 멍멍!
나비: 야옹~
```



다형성(polymorphism)

같은 `speak()` 를 불렀는데 객체 종류에 따라 다르게 동작했죠? 이렇게 '같은 명령, 다른 결과'를 **다형성**이라고 해요. 덕분에 `for a in [...]: a.speak()` 처럼 **종류를 신경 쓰지 않고** 일괄 처리할 수 있어 코드가 아주 유연해져요.

2.3 super() — 부모의 것도 챙기기

자식이 `__init__` 을 새로 만들면 부모의 `__init__` 이 안 불러요. 그러면 부모가 설정하던 속성이 사라지죠. 이때 `super()` 로 부모의 메서드를 명시적으로 불러 줘요. '부모가 하던 건 부모에게 맡기고, 나는 내 것만 추가'하는 방식이에요.

파이썬 코드

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, school):
        super().__init__(name, age)    # 부모의 __init__ 호출
        self.school = school           # 자식만의 속성 추가
    def info(self):
        print(f"{self.name}({self.age}세) - {self.school}")

s = Student("지우", 16, "한빛고")
s.info()
```

실행 결과

지우(16세) - 한빛고

`super().__init__(name, age)` 덕분에 `name`, `age` 는 부모가 챙기고, `Student` 는 `school` 만 추가하면 돼요. 부모 코드를 복사해 붙이지 않아도 되니 깔끔하죠.

2.4 캡슐화 — 함부로 못 건드리게 감추기

캡슐화(encapsulation)는 객체 내부의 데이터를 바깥에서 함부로 바꾸지 못하게 보호하는 거예요. 은행 잔액을 밖에서 아무나 `acc.balance = 999999` 라고 바꿔 버리면 큰일이잖아요. 파이썬은 이름 앞에 밑줄을 붙여 이 의도를 나타내요.

- `_name` (밑줄 하나): "내부용이니 밖에서 직접 건드리지 마세요"라는 약속. (강제는 아님)
- `__name` (밑줄 둘): 이름이 살짝 바뀌어(name mangling) 밖에서 접근이 실제로 막혀요.

파이썬 코드

```
class Account:
    def __init__(self, balance):
        self.__balance = balance    # 밀줄 돌: 감춤
    def deposit(self, amount):
        self.__balance += amount
    def get_balance(self):
        return self.__balance

acc = Account(1000)
acc.deposit(500)
print(acc.get_balance())
print(acc.__balance)    # 밖에서 직접 접근 시도
```

실행 결과

```
1500
AttributeError: 'Account' object has no attribute '__balance'
```

잔액은 `deposit()` 같은 정해진 통로로만 바꾸게 하고, 직접 접근은 막았어요. 마지막 줄에서 `acc.__balance` 로 몰래 꺼내려 하니 오류가 났죠. 이렇게 하면 데이터가 예상치 못하게 망가지는 걸 막을 수 있어요.

2.5 특수 메서드 — 파이썬 문법을 내 객체에 붙이기

`len(리스트)`, `print(값)` 같은 파이썬 기본 문법이 있죠. 이런 문법이 내가 만든 객체에도 통하게 만드는 게 특수 메서드예요. 이름이 `__이름__` 형태라 '던더(dunder) 메서드'라고도 불러요.

▶ `__str__` 과 `__repr__` — 객체를 예쁘게 출력

객체를 그냥 `print` 하면 `<__main__.Point object at 0x...>` 같은 암호가 나와요. `__str__` 을 정의하면 사람이 읽기 좋은 형태로 바꿀 수 있어요. `__repr__` 은 개발자용(리스트 안에 들어 있을 때 등)으로 쓰여요.

파이썬 코드

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return f"({self.x}, {self.y})"
    def __repr__(self):
        return f"Point({self.x}, {self.y})"

p = Point(3, 5)
print(p)                    # __str__ 사용
print([Point(1, 2), Point(3, 4)]) # 리스트 안: __repr__ 사용
```

실행 결과

```
(3, 5)
[Point(1, 2), Point(3, 4)]
```

▶ `__len__` — `len()` 이 통하게

`__len__` 을 정의하면 내 객체에도 `len()` 을 쓸 수 있어요.

파이썬 코드

```
class Team:
    def __init__(self, members):
        self.members = members
    def __len__(self):
        return len(self.members)

t = Team(["지우", "민수", "하나"])
print(len(t))
```

실행 결과

```
3
```

`len(t)` 를 부르면 파이썬이 뒤에서 `t.__len__()` 을 불러 줘요. `__eq__` (`==` 비교), `__add__` (`+` 연산) 등 다른 특수 메서드도 같은 원리예요.

특수 메서드	언제 불리나	무엇을 가능하게
<code>__init__</code>	객체 생성 시	초기 속성 설정
<code>__str__</code>	<code>print()</code> , <code>str()</code>	사람이 읽기 좋은 문자열
<code>__repr__</code>	리스트 안, 콘솔 표시	개발자용 표현
<code>__len__</code>	<code>len()</code>	길이 반환
<code>__eq__</code>	<code>==</code> 비교	같은지 비교 규칙

2.6 @property — 메서드를 속성처럼

`temp.fahrenheit()` 처럼 괄호를 붙여 부르는 대신, `temp.fahrenheit` 처럼 속성처럼 쓰고 싶을 때 `@property` 를 붙여요. 겉보기엔 속성인데 실제로는 계산이 돌아가는 거죠. 섭씨를 화씨로 자동 변환해 볼게요.

파이썬 코드

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def fahrenheit(self):
        return self._celsius * 9 / 5 + 32

temp = Temperature(25)
print(temp.fahrenheit)  # 괄호 없이 속성처럼!
```

실행 결과

77.0

`@property` 덕분에 `temp.fahrenheit` 라고만 써도 계산 결과가 나와요. 여기에 `@이름.setter` 를 찍지으면 값을 넣을 때 검사도 할 수 있어요. 예를 들어 가격에 음수가 들어오면 막는 식으로요.

파이썬 코드

```
class Product:
    def __init__(self, price):
        self._price = price
    @property
    def price(self):
        return self._price
    @price.setter
    def price(self, value):
        if value < 0:
            print("가격은 음수가 될 수 없습니다.")
        else:
            self._price = value

pr = Product(1000)
pr.price = -500    # setter 가 막아 줌
pr.price = 2000    # 정상 반영
print(pr.price)
```

실행 결과

```
가격은 음수가 될 수 없습니다.
2000
```



@property 는 캡슐화와 짝꿍이에요. 내부 값(`_price`)은 감추고, 읽고 쓰는 통로만 property 로 열어 주면 검사·가공을 끼워 넣을 수 있어요. 겉으로 그냥 속성처럼 쓰니 사용하는 쪽도 편하고요.

연습문제

1. `Animal` 을 상속한 `Bird` 클래스를 만들고 `speak()` 를 "짹짹"으로 오버라이딩해 보세요.
2. `Rectangle` 클래스에 `@property` 로 `area` (넓이=가로×세로)를 속성처럼 만들어 보세요.
3. `Point` 에 `__eq__` 를 추가해 두 점의 `x, y` 가 모두 같으면 `True` 가 나오게 해 보세요.

해답 · 힌트

② `@property` 아래 `def area(self): return self.w * self.h`. ③ `def __eq__(self, other): return self.x == other.x and self.y == other.y`.

컴프리헨션과 표현식

리스트를 만들 때마다 `result = []` 하고 `for` 돌면서 `append` 하는 코드, 1권에서 많이 썼죠? 파이썬에는 이걸 한 줄로 우아하게 줄이는 문법이 있어요. 바로 **컴프리헨션(comprehension)**이에요. 익숙해지면 '아, 이래서 파이썬 코드가 짧고 예쁘구나' 하고 느끼게 될 거예요.

이 장에서 배울 것

- 리스트 컴프리헨션으로 `for` 문 줄이기
- 조건 필터와 조건부 표현식
- 딕셔너리·셋 컴프리헨션
- 제너레이터 표현식으로 메모리 아끼기

3.1 리스트 컴프리헨션 — for 문을 한 줄로

먼저 1권 스타일 코드를 보고, 컴프리헨션으로 어떻게 줄어드는지 비교해 볼게요. 1부터 5까지 제곱을 담은 리스트를 만든다고 해요.

예전 방식

```
# 예전 방식
result = []
for x in range(1, 6):
    result.append(x * x)
print(result)
```

파이썬 코드

```
# 컴프리헨션 방식
squares = [x * x for x in range(1, 6)]
print(squares)
```

실행 결과

```
[1, 4, 9, 16, 25]
```

네 줄이 한 줄이 됐어요! 구조를 뜯어보면 **[표현식 for 변수 in 반복대상]** 예요. '`range(1, 6)` 의 각 `x` 에 대해 `x * x` 를 모아 리스트로 만들라'는 뜻이죠. 읽는 순서는 뒤(for)부터예요: '`x` 를 하나씩 꺼내서 $\rightarrow x*x$ 를 만들어 담는다'.



컴프리헨션 읽는 법

```
[x * x for x in range(1, 6)]
```

↑ 답을 값 ↑ 하나씩 꺼내기

"각 x 마다, x*x 를 담아라" 라고 소리 내어 읽으면 익숙해져요.

3.2 조건 붙이기 — 필터링

뒤에 `if 조건` 을 붙이면 `조건을 만족하는 것만` 담아요. 1~10 중 짝수만 골라 볼게요.

파이썬 코드

```
evens = [x for x in range(1, 11) if x % 2 == 0]
print(evens)
```

실행 결과

```
[2, 4, 6, 8, 10]
```

`x % 2 == 0` (2로 나눈 나머지가 0, 즉 짝수)인 `x` 만 통과시켜 담았어요. 값을 변형하는 것도 자유예요. 단어 리스트를 모두 대문자로 바꾸거나, 길이를 뽑을 수도 있죠.

파이썬 코드

```
words = ["hello", "world", "python"]
upper = [w.upper() for w in words]
lengths = [len(w) for w in words]
print(upper)
print(lengths)
```

실행 결과

```
['HELLO', 'WORLD', 'PYTHON']
[5, 5, 6]
```

3.3 조건부 표현식 — 한 줄 if/else

값 자체를 조건에 따라 다르게 `고르고` 싶을 땐 `A if 조건 else B` 를 써요. 이건 컴프리헨션 전용이 아니라 파이썬 전체에서 쓰는 문법이에요. 각 숫자에 '짝수/홀수' 라벨을 붙여 볼게요.

파이썬 코드

```
nums = [1, 2, 3, 4, 5]
labels = ["짝수" if n % 2 == 0 else "홀수" for n in nums]
print(labels)
```

실행 결과

```
['홀수', '짝수', '홀수', '짝수', '홀수']
```

⚠ if 의 위치에 주의!

- 필터링(고를 때)의 `if` 는 뒤에: `[x for x in a if x > 0]`
 - 값 고르기(if/else)의 `if` 는 앞에: `[x if x > 0 else 0 for x in a]`
- 필터는 'else 없이 뒤', 값 고르기는 'else 있고 앞'. 헷갈리기 쉬우니 꼭 기억하세요.

3.4 딕셔너리·셋 컴프리헨션

대괄호 `[]` 대신 중괄호 `{}` 를 쓰면 딕셔너리나 셋도 만들 수 있어요. 딕셔너리는 `{키: 값 for ...}` 형태예요.

파이썬 코드

```
squares_dict = {x: x * x for x in range(1, 6)}
print(squares_dict)

fruits = ["사과", "바나나", "포도"]
prices = [1000, 3000, 5000]
menu = {f: p for f, p in zip(fruits, prices)}
print(menu)
```

실행 결과

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{'사과': 1000, '바나나': 3000, '포도': 5000}
```

`zip` 은 두 리스트를 짝지어 주는 함수예요(10장에서 더 다뤄요). 여기서 과일과 가격을 짝지어 메뉴 딕셔너리를 한 줄로 만들었죠. 셋 컴프리헨션은 `키:값` 없이 값만 쓰면 되고, `중복이 자동 제거`돼요.

파이썬 코드

```
nums = [1, 2, 2, 3, 3, 3, 4]
unique_squares = {x * x for x in nums}
print(unique_squares)
```

실행 결과

```
{16, 1, 4, 9}
```



셋은 중복을 없애고 `순서를 보장하지` 않아요. 그래서 출력이 `{16, 1, 4, 9}` 처럼 우리가 넣은 순서와 다르게 보일 수 있어요. 이건 오류가 아니라 셋의 정상 동작이에요.

3.5 제너레이터 표현식 — 메모리를 아끼는 형제

대괄호 대신 **소괄호** `()` 를 쓰면 **제너레이터 표현식**이 돼요. 리스트 컴프리헨션은 결과를 **전부 메모리에 만들어** 두지만, 제너레이터는 **필요할 때 하나씩** 만들어 내요. 큰 데이터를 다룰 때 메모리를 크게 아낄 수 있죠 (자세한 원리는 4장에서!).

파이썬 코드

```
gen = (x * x for x in range(1, 6))
print(gen)           # 리스트가 아니라 제너레이터 객체

# sum 처럼 한 번 훑기만 하면 되는 경우 특히 좋아요
print(sum(x * x for x in range(1, 101))) # 1^2+...+100^2
```

실행 결과

```
<generator object <genexpr> at 0x...>
338350
```

`sum()` 같은 함수에 넘길 땐 **괄호도 생략**할 수 있어요(`sum(x * x for x in range(1, 101))`). 1부터 100까지 제곱의 합을 **리스트를 만들지 않고** 계산했죠. '전부 담을 필요 없이 훑기만 하면 될 때'는 제너레이터가 정답이에요.

3.6 중첩 컴프리헨션

`for` 를 두 번 쓰면 이중 반복도 한 줄로 표현돼요. 다만 너무 복잡하면 오히려 읽기 어려우니 **2단까지만** 권해요.

파이썬 코드

```
pairs = [(x, y) for x in range(1, 3) for y in range(1, 3)]
print(pairs)
```

실행 결과

```
[(1, 1), (1, 2), (2, 1), (2, 2)]
```

바깥 `for x` 가 먼저, 안쪽 `for y` 가 그 안에서 도는 거예요. 일반 중첩 `for` 문을 쓸 때와 순서가 같아요.

연습문제

- 1~20 중 3의 배수만 담은 리스트를 컴프리헨션으로 만들어 보세요.
- 문자열 리스트 `["apple", "banana", "cherry"]` 에서 길이가 6 이상인 단어만 골라 보세요.
- `{"a": 1, "b": 2, "c": 3}` 의 값을 2배로 만든 새 딕셔너리를 컴프리헨션으로 만들어 보세요.

해답 · 힌트

① `[x for x in range(1, 21) if x % 3 == 0]` . ② `[w for w in words if len(w) >= 6]` . ③ `{k: v * 2 for k, v in d.items()}` .

이터레이터와 제너레이터

3장 끝에서 만난 제너레이터, 궁금했죠? 이 장에서는 `for` 문이 뒤에서 어떻게 돌아가는지, 그리고 **필요할 때마다 하나씩 값을 꺼내 주는** 제너레이터를 직접 만드는 법을 배워요. 자판기에 비유하면, 음료수를 한꺼번에 다 쏟아내는 게 아니라 버튼을 누를 때마다 한 개씩 내주는 거예요. 메모리를 아끼는 파이썬의 비밀 무기죠.

이 장에서 배울 것

- iterable 과 iterator 의 차이
- `iter()` 와 `next()` 의 동작
- `yield` 로 제너레이터 만들기
- 메모리 효율과 무한 시퀀스

4.1 iterable 과 iterator

리스트, 문자열, 딕셔너리처럼 `for` 로 하나씩 꺼낼 수 있는 것들을 **iterable(반복 가능한 것)**이라고 해요. 그리고 실제로 '다음 값'을 하나씩 꺼내 주는 장치를 **iterator(반복자)**라고 하죠. 비유하면 iterable 은 '음료수가 든 자판기', iterator 는 '버튼을 눌러 하나씩 꺼내는 그 동작'이에요.

4.2 iter() 와 next() — for 문의 속사정

`for` 문은 사실 뒤에서 `iter()` 로 iterator 를 얻고, `next()` 로 값을 하나씩 꺼내고 있었어요. 직접 손으로 돌려 볼게요.

파이썬 코드

```
nums = [10, 20, 30]
it = iter(nums)      # iterator 얻기
print(next(it))      # 다음 값 꺼내기
print(next(it))
print(next(it))
print(next(it))      # 더 없으면 StopIteration
```

실행 결과

```
10
20
30
StopIteration 발생
```

`next()` 를 부를 때마다 다음 값이 나오고, 값이 **동나면** `StopIteration` 이라는 신호를 내요. `for` 문은 이 신호를 받으면 '아, 다 꺼냈구나' 하고 조용히 반복을 멈춰요. 즉 `for x in nums:` 는 이 과정을 자동으로 해 주는 거였죠.



핵심

`for` 문 = `iter()` 로 반복자 얻기 + `next()` 반복 호출 + `StopIteration` 만나면 멈추기. 이걸 파이썬이 알아서 해 줘요.

4.3 제너레이터 — `yield` 로 만드는 자판기

이제 우리가 직접 '값을 하나씩 내주는 장치'를 만들어 봐요. 함수 안에서 `return` 대신 `yield` 를 쓰면, 그 함수는 **제너레이터 (generator)**가 돼요. `yield` 는 '값을 하나 내주고, 여기서 잠깐 멈춰 있다가, 다음에 부르면 이어서 실행'하는 특별한 명령이에요.

파이썬 코드

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i      # 값을 하나 내주고 잠깐 멈춤
        i += 1

for num in count_up_to(3):
    print(num)
```

실행 결과

```
1
2
3
```

`yield i` 를 만나면 `i` 값을 하나 내주고 그 자리에서 **멈춰요**. 다음 값을 요청하면(다음 `for` 반복) 멈췄던 `i += 1` 부터 이어서 실행되죠. `return` 은 함수를 아예 끝내지만, `yield` 는 '잠깐 멈춤'이라는 게 결정적 차이예요.

`next()` 로 직접 하나씩 꺼내 볼 수도 있어요.

파이썬 코드

```
def simple_gen():
    yield "첫번째"
    yield "두번째"
    yield "세번째"

g = simple_gen()
print(next(g))
print(next(g))
print(next(g))
```

실행 결과

```
첫번째
두번째
세번째
```

`yield` 를 만날 때마다 그 값을 내주고 멈추니, `next()` 를 부를 때마다 차례로 하나씩 나와요. 딱 자판기죠. 버튼(`next`)을 누를 때마다 음료수(값) 하나!

4.4 왜 제너레이터인가 — 메모리 효율

리스트는 값을 전부 미리 만들어 메모리에 쌓아 뒹요. 100만 개면 100만 개를 다 담죠. 반면 제너레이터는 요청할 때 하나만 만들고 버려요. 그래서 데이터가 아주 크거나, 끝이 없을 때 큰 힘을 발휘해요.

파이썬 코드

```
def squares_gen(n):
    for i in range(1, n + 1):
        yield i * i

gen = squares_gen(5)
print(list(gen))  # 필요하면 list() 로 한꺼번에 받을 수도
```

실행 결과

```
[1, 4, 9, 16, 25]
```



제너레이터는 한 번 다 꺼내면 비어요. 자판기 음료를 다 뽑으면 빈 것과 같죠. 다시 쓰려면 제너레이터를 새로 만들어야 해요. 그래서 `list(gen)` 을 두 번 하면 두 번째엔 빈 리스트가 나와요.

4.5 무한 시퀀스 — 끝없이 만들어 내기

리스트로는 '끝없는 수열'을 만들 수 없어요(메모리가 터지니까). 하지만 제너레이터는 `while True` 로 무한히 값을 내줄 수 있어요. 필요한 만큼만 꺼내 쓰면 되죠.

파이썬 코드

```
def infinite_counter():
    n = 1
    while True:          # 무한 반복이지만 괜찮아요
        yield n
        n += 1

counter = infinite_counter()
result = []
for c in counter:
    if c > 5:
        break           # 원하는 만큼만 꺼내고 멈춤
    result.append(c)
print(result)
```

실행 결과

```
[1, 2, 3, 4, 5]
```

무한 반복인데도 프로그램이 멈추지 않아요. 제너레이터는 값을 미리 다 만들지 않고 필요할 때만 하나씩 내주기 때문이죠. 우리가 `break` 로 멈출 때까지만 돌아요.

고전 예제인 피보나치 수열도 제너레이터로 우아하게 표현돼요.

파이썬 코드

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b    # 다음 두 값으로 갱신

fib = fibonacci()
first_ten = []
for _ in range(10):
    first_ten.append(next(fib))
print(first_ten)
```


실행 결과

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

`a, b = b, a + b` 는 1권에서 배운 '동시 할당'이에요. 이전 두 수를 더해 다음 수를 만드는 피보나치 규칙을 한 줄로 표현했죠. 무한히 이어지는 수열을 `next()` 로 필요한 만큼만 꺼내 쓰는 게 제너레이터의 매력이에요.

연습문제

1. 1부터 `n` 까지 짝수만 내주는 제너레이터 `even_gen(n)` 을 만들어 보세요.
2. 리스트를 받아 거꾸로 하나씩 내주는 제너레이터를 만들어 보세요. (힌트: `reversed()` 또는 인덱스)
3. 무한히 `2, 4, 8, 16, ...` (2의 거듭제곱)을 내주는 제너레이터를 만들고 앞 5개만 뽑아 보세요.

해답 · 힌트

① `for i in range(2, n+1, 2): yield i`. ③ `n = 2; while True: yield n; n *= 2` → `next()` 5번 하면 `[2, 4, 8, 16, 32]`.

함수를 자유자재로

1권에서 함수는 '코드에 이름 붙이기'였죠. 파이썬에서 함수는 그 이상이에요. 함수를 변수에 담고, 다른 함수에 넘기고, 함수가 함수를 만들어 내기까지 해요. 이걸 이해하면 `map` · `filter` · `sorted` 같은 도구를 손발처럼 쓰고, 다음 장의 데코레이터도 술술 풀려요.

이 장에서 배울 것

- 함수는 값이다 — 일급 함수
- 람다(`lambda`) — 이름 없는 짧은 함수
- `map` · `filter` · `sorted` 의 `key`
- 클로저(closure) — 함수가 값을 기억하기

5.1 함수는 값이다 — 일급 함수

파이썬에서 함수는 숫자나 문자열처럼 하나의 값이에요. 그래서 변수에 담을 수도, 다른 함수에 인자로 넘길 수도 있죠. 이런 성질을 '함수가 일급(first-class)이다'라고 해요.

● ● ● 파이썬 코드

```
def square(x):
    return x * x

f = square      # 함수를 변수에 담기(괄호 없이!)
print(f(5))     # f 로 부를 수 있어요

def apply(func, value):    # 함수를 인자로 받기
    return func(value)

print(apply(square, 4))
```

실행 결과

```
25
16
```

⚠ 괄호에 주의!

`f = square` 는 함수 자체를 담는 거예요(괄호 없음). `f = square(5)` 라고 하면 함수를 실행한 결과값 25가 담겨요. '함수를 넘길 때는 괄호를 떼고, 실행할 때만 괄호를 붙인다'고 기억하세요.

5.2 람다 — 이름 없는 한 줄 함수

아주 간단한 함수를 `def` 없이 한 줄로 만드는 게 람다(lambda)예요. **lambda 매개변수: 반환값** 형태죠. 잠깐 쓰고 버릴 짧은 함수에 딱이에요.

파이썬 코드

```
double = lambda x: x * 2
print(double(10))

add = lambda a, b: a + b
print(add(3, 7))
```

실행 결과

```
20
10
```

`lambda x: x * 2` 는 `def double(x): return x * 2` 와 똑같아요. 다만 이름을 안 붙이고 그 자리에서 바로 쓸 수 있죠. 람다는 `return` 을 쓰지 않아요 — 콜론 뒤 표현식이 곧 반환값이에요.



람다는 짧을 때만 쓰세요. 로직이 길어지면 그냥 `def` 로 이름 붙인 함수가 훨씬 읽기 좋아요. 람다의 진가는 다음에 나올 `map · sorted` 같은 곳에서 '일회용 함수'로 넘길 때 나와요.

5.3 map · filter — 일괄 변형과 걸러내기

`map` 은 모든 값에 함수를 일괄 적용하고, `filter` 는 조건에 맞는 것만 걸러내요. 3장 컴프리헨션과 비슷한 일을 하는데, 함수를 넘긴다는 게 특징이에요.

파이썬 코드

```
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))
print(doubled)

nums2 = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums2))
print(evens)
```

실행 결과

```
[2, 4, 6, 8]
[2, 4, 6]
```

`map(함수, 반복대상)` 은 각 값에 함수를 적용한 결과를, `filter(함수, 반복대상)` 은 함수가 `True` 를 돌려주는 값만 남겨요. 둘 다 결과가 바로 리스트가 아니라서 `list()` 로 감싸 확인했어요. (사실 이런 건 컴프리헨션으로 써도 되지만, `map` / `filter` 도 코드에서 자주 보여요.)

5.4 sorted 의 key — 정렬 기준 정하기

`sorted()` 에 `key=함수` 를 주면 무엇을 기준으로 정렬할지 직접 정할 수 있어요. 이게 정렬의 진짜 실력이에요. 단어를 길이 순으로 정렬해 볼게요.

파이썬 코드

```
words = ["banana", "kiwi", "apple", "fig"]
by_length = sorted(words, key=len)
print(by_length)
```

실행 결과

```
['fig', 'kiwi', 'apple', 'banana']
```

`key=len` 은 '각 단어의 `len()` (길이)을 기준으로 정렬하라'는 뜻이에요. 여기서도 함수를 값처럼 넘기고 있죠(일급 함수!). 이번엔 (이름, 점수) 튜플을 점수 높은 순으로 정렬해 봐요. 기준을 람다로 꼭 집어요.

파이썬 코드

```
students = [("지우", 85), ("민수", 92), ("하나", 78)]
by_score = sorted(students, key=lambda s: s[1], reverse=True)
print(by_score)
```

실행 결과

```
[('민수', 92), ('지우', 85), ('하나', 78)]
```

`key=lambda s: s[1]` 은 '각 튜플의 두 번째 값(점수)을 기준으로 삼으라'는 뜻이에요. `reverse=True` 는 내림차순(큰 것 부터). 이 한 줄로 '점수 높은 순 정렬'이 끝나요. 람다가 여기서 얼마나 편리한지 느껴지죠?

5.5 클로저 — 함수가 값을 기억하기

함수 안에서 함수를 만들어 돌려주면, 안쪽 함수는 바깥 함수의 변수를 기억해요. 이 '기억하는 함수'를 클로저(closure)라고 해요. 붕어빵 틀에 '×3 하는 도장'을 새겨 찍어내는 것처럼, 만들 때의 설정을 품고 다니는 함수죠.

파이썬 코드

```
def make_multiplier(n):
    def multiplier(x):
        return x * n        # 바깥의 n 을 기억!
    return multiplier

times3 = make_multiplier(3)
times5 = make_multiplier(5)
print(times3(10))
print(times5(10))
```

실행 결과

```
30
50
```

`make_multiplier(3)` 이 돌려준 `times3` 은 `n=3` 을 기억하고 있어요. `times5` 는 `n=5` 를 기억하고요. 같은 틀에서 나왔지만 각자 다른 설정을 품은 맞춤 함수가 만들어진 거죠. 이게 클로저이고, 다음 장 데코레이터의 뼈대이기도 해요.

바깥 변수를 수정하고 싶다면 `nonlocal` 을 써요. 호출될 때마다 숫자를 세는 카운터를 만들어 볼게요.

파이썬 코드

```
def make_counter():
    count = 0
    def counter():
        nonlocal count        # 바깥 count 를 수정하겠다는 선언
        count += 1
        return count
    return counter

c = make_counter()
print(c())
print(c())
print(c())
```

실행 결과

```
1
2
3
```

`c()` 를 부를 때마다 `count` 가 기억되고 1씩 늘어나요. `nonlocal count` 가 없으면 안쪽에서 새 지역 변수를 만들려다 오류가 나요(1권의 지역/전역 변수 규칙 때문이에요). '바깥 변수를 바꿀 거야'라고 선언해 주는 게 `nonlocal` 이예요.

연습문제

- 리스트 `[3, -1, 4, -5, 2]` 를 `sorted` 와 `key` 로 절댓값 기준 정렬해 보세요. (힌트: `key=abs`)
- `filter` 와 람다로 문자열 리스트에서 'a'로 시작하는 단어만 골라 보세요.
- `make_adder(n)` 클로저를 만들어 `add10 = make_adder(10)` 이 `add10(5) == 15` 가 되게 해 보세요.

해답 · 힌트

① `sorted(nums, key=abs)`. ② `list(filter(lambda w: w.startswith("a"), words))`. ③ `def make_adder(n): return lambda x: x + n`.

데코레이터

데코레이터는 이름부터 '장식'이에요. **기존 함수를 건드리지 않고, 그 앞뒤에 기능을 덧입히는** 도구죠. 선물 포장에 비유하면 완벽해요 — 안의 선물(원래 함수)은 그대로 두고, 예쁜 포장지(부가 기능)로 감싸는 거예요. 실행 시간 측정, 로그 남기기, 권한 검사 같은 걸 함수마다 복붙하지 않고 `@` 한 줄로 붙일 수 있어요. 5장 클로저를 알면 어렵지 않아요.

이 장에서 배울 것

- 함수를 감싸는 함수 (선물 포장 비유)
- `@` 문법의 정체
- 인자 있는 함수도 감싸기 (`*args`, `**kwargs`)
- 실행시간 측정·로깅 데코레이터와 `functools.wraps`

6.1 함수를 감싸는 함수

데코레이터는 **함수를 받아서, 감싼 새 함수를 돌려주는 함수**예요. 말이 어렵지, 코드를 보면 5장의 클로저와 똑같은 구조예요. 원래 함수 실행 **전후**에 뭔가를 끼워 넣는 걸 만들어 볼게요.

파이썬 코드

```
def my_decorator(func):
    def wrapper():
        print("함수 실행 전")
        func()          # 원래 함수 실행
        print("함수 실행 후")
    return wrapper

@my_decorator
def say_hello():
    print("안녕하세요!")

say_hello()
```

실행 결과

```
함수 실행 전
안녕하세요!
함수 실행 후
```

`say_hello()` 를 불렀을 뿐인데 앞뒤로 메시지가 붙었죠? `@my_decorator` 가 `say_hello` 를 포장지(wrapper)로 감쌌기 때문이에요. 안의 선물(`say_hello`)은 그대로인데, 포장지가 '실행 전/후' 문구를 더해 준 거예요.

6.2 @ 문법의 정체

`@my_decorator` 는 사실 편의 문법이에요. 아래 두 코드는 완전히 같아요.

● ● ● @ 문법 = 이것의 축약

```
@my_decorator
def say_hello():
    print("안녕하세요!")

# ↑ 위는 아래를 짧게 쓴 것일 뿐

def say_hello():
    print("안녕하세요!")

say_hello = my_decorator(say_hello)  # 감싼 걸 다시 이름에 담기
```



데코레이터 한 줄 요약

`@데코레이터` 는 '이 함수를 데코레이터로 감싸서, 같은 이름에 다시 담아라'는 뜻이에요. 그래서 이후 `say_hello()` 를 부르면 감싼 버전이 실행돼요.

6.3 인자 있는 함수도 감싸기 — *args, **kwargs

실전 함수는 대부분 인자를 받죠. 포장지가 어떤 인자든 받아 원래 함수에 그대로 넘겨주게 하려면 `*args, **kwargs` 를 써요(1권·10장 참고). '인자가 몇 개든, 이름 붙은 인자든 다 받아서 그대로 전달'하는 만능 포장지예요.

파이썬 코드

```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"[로그] {func.__name__} 호출됨")
        result = func(*args, **kwargs)    # 인자 그대로 전달
        print(f"[로그] 결과: {result}")
        return result                    # 결과도 그대로 돌려줌
    return wrapper

@logger
def add(a, b):
    return a + b

add(3, 5)
```

실행 결과

```
[로그] add 호출됨
[로그] 결과: 8
```

`func.__name__` 은 함수의 이름(여기선 "add")을 알려 줘요. 포장지가 인자를 그대로 넘기고(`*args, **kwargs`), 결과도 `return` 으로 돌려주니, 원래 함수의 동작은 하나도 안 망가지고 로그만 추가됐어요. 이제 어떤 함수든 `@logger` 한 줄만 붙이면 자동 로깅이 돼요.

6.4 실전 — 실행 시간 측정

데코레이터의 가장 인기 있는 활용이 '이 함수가 얼마나 걸렸나' 재는 거예요. `time` 모듈로 시작·끝 시각을 재서 차이를 출력해요.

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'{func.__name__} 실행시간: {end - start:.4f}초')
        return result
    return wrapper

@timer
def slow_sum(n):
    total = 0
    for i in range(n):
        total += i
    return total

print(slow_sum(1000000))
```

실행 결과

```
slow_sum 실행시간: 0.0367초
499999500000
```



실행시간(0.0367초)은 컴퓨터·상황마다 매번 달라져요. 여러분이 돌리면 다른 숫자가 나오는 게 정상이에요. 중요한 건 '함수를 안 고치고도 측정 기능을 붙였다'는 점이에요. 성능을 확인하고 싶은 함수에 `@timer` 만 붙이면 되죠.

6.5 functools.wraps — 원래 이름표 지키기

데코레이터로 감싸면 함수의 이름·설명이 포장지 것으로 바뀌는 부작용이 있어요. `func.__name__` 이 "wrapper"로 나와 버리죠. 이걸 막으려면 포장지 위에 `@functools.wraps(func)` 를 붙여요. '원래 함수의 이름표를 그대로 유지하라'는 뜻이에요.

```
import functools

def my_deco(func):
    @functools.wraps(func)      # 원래 이름·docstring 유지
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@my_deco
def greet():
    """인사하는 함수"""
    return None

print(greet.__name__)      # wraps 덕분에 원래 이름
print(greet.__doc__)      # 설명(docstring)도 유지
```

실행 결과

```
greet
인사하는 함수
```



실무 습관

데코레이터를 만들 땐 거의 항상 `@functools.wraps(func)` 를 붙이세요. 안 붙이면 함수 이름·설명이 죄다 "wrapper"로 바뀌어 나중에 디버깅할 때 헛갈려요. '데코레이터 만들면 wraps 도 세트'로 외워 두면 좋아요.

연습문제

- 함수 실행 전에 "==== 시작 =====", 후에 "==== 끝 =====" 를 출력하는 데코레이터를 만들어 아무 함수에 붙여 보세요.
- 위 `logger` 를 참고해, 함수가 받은 인자(`args`)도 함께 출력하도록 바꿔 보세요.
- `@functools.wraps` 를 뺐을 때와 넣었을 때 `함수.__name__` 이 어떻게 달라지는지 직접 확인해 보세요.

해답 · 힌트

① `def wrapper(*a, **k): print("==== 시작 ====="); r = func(*a, **k); print("==== 끝 ====="); return r`. ② `wrapper` 안에서 `print("인자:", args)` 추가. ③ `wraps` 없으면 `wrapper`, 있으면 원래 함수명이 나와요.

정규표현식(re)

'전화번호만 뽑아줘', '이메일 형식이 맞는지 검사해줘' 같은 일, 조건문으로 하려면 코드가 엄청 길어져요. **정규표현식**은 문자열의 패턴을 짧은 기호로 표현해 찾고·검사하고·바꾸는 강력한 도구예요. 처음엔 기호가 암호처럼 보이지만, 자주 쓰는 몇 개만 알아도 실무의 8할은 해결돼요. 파이썬은 **re** 모듈로 제공해요.

이 장에서 배울 것

- 패턴 매칭이 뭔지, 자주 쓰는 메타문자
- **search** — 패턴 찾기
- **findall** — 모두 찾기
- **sub** — 찾아서 바꾸기 (마스킹 등)

7.1 자주 쓰는 메타문자

정규표현식은 글자 대신 '패턴'을 적는 언어예요. 예를 들어 '숫자 3개'를 **333** 이라고 꼭 집지 않고 '**\d** (숫자)가 3번'이라고 표현하죠. 가장 자주 쓰는 것만 추렸어요.

기호	의미	예
\d	숫자 하나 (0~9)	\d\d → "42"
\w	글자·숫자·밑줄 하나	\w+ → "hong_1"
.	아무 문자 하나	a.c → "abc", "a1c"
+	앞 패턴 1번 이상 반복	\d+ → "12345"
*	앞 패턴 0번 이상 반복	a* → "", "aaa"
{n}	앞 패턴 정확히 n번	\d{4} → "2026"
[...]	괄호 안 문자 중 하나	[a-z] → 소문자



파이썬에선 패턴 문자열 앞에 **r** 을 붙여요(예: **r"\d+"**). 이걸 **raw 문자열**이라고 하는데, **\d** 의 역슬래시가 엉뚱하게 해석되는 걸 막아 줘요. '정규식은 **r""** 로 감싼다'고 습관 들이세요.

7.2 search — 패턴이 있는지 찾기

re.search(패턴, 문자열) 은 문자열 어딘가에 패턴이 있으면 그 부분을 찾아 줘요. 문장 속에서 전화번호를 뽑아 볼게요.

파이썬 코드

```
import re

text = "제 전화번호는 010-1234-5678 입니다."
m = re.search(r"\d{3}-\d{4}-\d{4}", text)
print(m.group())
```

실행 결과

010-1234-5678

`\d{3}-\d{4}-\d{4}` 는 '숫자 3개 - 숫자 4개 - 숫자 4개' 라는 전화번호 패턴이에요. 찾으면 `match` 객체가 나오고, `m.group()` 으로 실제 찾은 문자열을 꺼내요. 못 찾으면 `None` 이 나와요.

파이썬 코드

```
m2 = re.search(r"\d{3}-\d{4}-\d{4}", "번호 없음")
print(m2)
```

실행 결과

None



None 확인은 필수

패턴을 못 찾으면 `search` 는 `None` 을 돌려줘요. 그 상태에서 `m2.group()` 을 부르면 오류가 나요. 실무에선 `if m:` 로 찾았는지 먼저 확인하고 `group()` 을 부르세요.

7.3 findall — 모두 찾기

`re.findall(패턴, 문자열)` 은 패턴에 맞는 걸 전부 찾아 리스트로 줘요. 가격 문장에서 숫자만 몽땅 뽑아 볼게요.

파이썬 코드

```
import re

text = "가격: 사과 1000원, 배 2500원, 포도 5000원"
prices = re.findall(r"\d+", text)
print(prices)
```

실행 결과

['1000', '2500', '5000']

`\d+` 는 '숫자가 1개 이상 연달아' 있는 덩어리예요. 문장 속 숫자 세 덩어리를 한 번에 다 잡았죠. 참고로 결과는 **문자열** 리스트예요 — 숫자로 계산하려면 `int()` 로 바꿔야 해요.

7.4 sub — 찾아서 바꾸기 (마스킹)

`re.sub(패턴, 바꿀문자열, 원본)` 은 패턴에 맞는 부분을 다른 문자열로 교체해요. 개인정보 마스킹(가리기)에 딱이에요. 주민번호를 별표로 가려 볼게요.

파이썬 코드

```
import re

text = "이름: 홍길동, 주민번호: 901010-1234567"
masked = re.sub(r"\d{6}-\d{7}", "*****-*****", text)
print(masked)
```

실행 결과

```
이름: 홍길동, 주민번호: *****-*****
```

주민번호 패턴(`\d{6}-\d{7}`)을 찾아 별표로 바꿨어요. 이름은 그대로 두고 민감한 번호만 정확히 가렸죠. 로그·화면에서 개인 정보를 숨길 때 이 방식을 아주 많이 써요.

7.5 실전 — 이메일·전화번호 추출과 그룹

여러 이메일을 한꺼번에 뽑아 볼게요. 이메일 패턴은 '@' 앞뒤에 글자들'로 잡아요.

파이썬 코드

```
import re

text = "문의: hong@test.com, support@example.co.kr"
emails = re.findall(r"[a-zA-Z0-9._]+@[a-zA-Z0-9._]+", text)
print(emails)
```

실행 결과

```
['hong@test.com', 'support@example.co.kr']
```

`[a-zA-Z0-9._]+` 는 '영문·숫자·점·밑줄이 1개 이상', 그다음 '@', 다시 도메인 부분이에요. 실무의 이메일 패턴은 더 엄격하게 쓰이지만, 이 정도면 대부분 잡혀요.

괄호 `()` 로 패턴을 **그룹**으로 묶으면, 매칭된 부분을 조각조각 나눠 꺼낼 수 있어요. 날짜에서 연·월·일을 따로 빼 볼게요.

파이썬 코드

```
import re

m = re.search(r"(\d{4})-(\d{2})-(\d{2})", "오늘은 2026-08-11 입니다.")
print(m.group())      # 전체
print(m.group(1))     # 첫 번째 그룹
print(m.group(2))     # 두 번째 그룹
print(m.group(3))     # 세 번째 그룹
```

실행 결과

```
2026-08-11
2026
08
11
```

`group(0)` 또는 `group()` 은 전체, `group(1)` 부터는 괄호로 묶은 순서대로 조각을 줘요. 날짜에서 연도만, 월만 딱딱 빼낼 수 있죠. 복잡한 문자열에서 필요한 부분만 정확히 분해할 때 그룹이 아주 유용해요.



정규식 3대 함수 요약

- `re.search` : 처음 하나 찾기 → `match` 객체 or `None`
- `re.findall` : 전부 찾기 → 리스트
- `re.sub` : 찾아서 바꾸기 → 바뀐 문자열

연습문제

1. 문장 "주문번호 A1234, B5678, C9012" 에서 주문번호(영문1+숫자4)를 `findall` 로 모두 뽑아 보세요. (힌트: `[A-Z]\d{4}`)
2. 전화번호 010-1234-5678 의 가운데 네 자리를 `****` 로 마스킹해 보세요. (힌트: `sub` 와 그룹)
3. 문자열에서 이메일이 있으면 "이메일 발견", 없으면 "없음"을 출력하도록 `if m:` 를 활용해 보세요.

해답 · 힌트

① `re.findall(r"[A-Z]\d{4}", text)`. ② `re.sub(r"(\d{3})-(\d{4})-(\d{4})", r"\1-****-\2", tel)`
— `\1` 은 첫 그룹. ③ `m = re.search(패턴, text); print("이메일 발견" if m else "없음")`.

유용한 표준 라이브러리

파이썬이 사랑받는 큰 이유 중 하나는 '배터리 포함(batteries included)' — 설치만 하면 쓸 수 있는 강력한 **표준 라이브러리**가 잔뜩 들어 있다는 거예요. 바퀴를 새로 발명할 필요 없이, 남이 잘 만들어 둔 도구를 `import` 만 하면 되죠. 이 장에선 실무에서 손이 자주 가는 것만 골라 소개해요. 이것만 알아도 코드가 확 짧아져요.

이 장에서 배울 것

- `collections` — Counter · defaultdict · deque
- `itertools` — 조합·순열·무한 카운터
- `functools` — reduce
- `pathlib` 와 `json`

8.1 collections.Counter — 개수 세기 끝판왕

리스트에서 각 항목이 몇 번 나오는지 세는 일, 정말 자주 하죠. `Counter` 는 이걸 **한 줄로** 끝내요. 과일 등장 횟수를 세어 볼게요.

파이썬 코드

```
from collections import Counter

words = ["사과", "배", "사과", "포도", "사과", "배"]
count = Counter(words)
print(count)
print(count["사과"])          # 특정 항목 개수
print(count.most_common(2))   # 가장 많은 2개
```

실행 결과

```
Counter({'사과': 3, '배': 2, '포도': 1})
3
[('사과', 3), ('배', 2)]
```

`Counter(리스트)` 만으로 각 항목의 개수를 세어 딕셔너리처럼 만들어 줘요. `most_common(n)` 은 **많이 나온 순**으로 상위 n 개를 뽑아 주죠. '가장 인기 있는 상품 TOP 3' 같은 걸 순식간에 구할 수 있어요.

8.2 defaultdict — 없는 키도 알아서 준비

일반 딕셔너리는 없는 키에 바로 접근하면 오류가 나요. 그래서 '키가 있으면 더하고 없으면 새로 만들고'를 매번 확인해야 했죠. `defaultdict` 는 없는 키를 자동으로 기본값과 함께 만들어 줘 이 번거로움을 없애요.

파이썬 코드

```
from collections import defaultdict

d = defaultdict(list)      # 없는 키는 자동으로 빈 리스트로 시작
d["과일"].append("사과")
d["과일"].append("배")
d["채소"].append("당근")
print(dict(d))
```

실행 결과

```
{'과일': ['사과', '배'], '채소': ['당근']}
```

`defaultdict(list)` 는 '처음 보는 키는 빈 리스트로 시작하라'는 뜻이에요. 그래서 `d["과일"]` 가 없어도 바로 `.append()` 할 수 있죠. 항목을 카테고리별로 묶을 때 특히 편해요. (숫자 세기엔 `defaultdict(int)`)

8.3 deque — 양쪽으로 빠른 큐

리스트는 앞쪽에 넣고 빼는 게 느려요(뒤로 다 밀어야 하니까). `deque` (덱)는 양쪽 끝 모두 빠르게 넣고 뺄 수 있어요. 대기열, 최근 기록 관리 등에 제격이에요.

파이썬 코드

```
from collections import deque

q = deque([1, 2, 3])
q.append(4)      # 오른쪽에 추가
q.appendleft(0)  # 왼쪽에 추가
print(q)
print(q.popleft()) # 왼쪽에서 꺼내기
print(q.pop())    # 오른쪽에서 꺼내기
print(q)
```

실행 결과

```
deque([0, 1, 2, 3, 4])
0
4
deque([1, 2, 3])
```

`appendleft` · `popleft` 로 왼쪽 끝을 다루는 게 리스트에 없는 힘이에요. '먼저 들어온 게 먼저 나가는' 대기열(큐)을 만들 때 `append` (뒤로 넣기) + `popleft` (앞에서 빼기) 조합을 많이 써요.

8.4 itertools — 조합·순열·무한 카운터

`itertools` 는 반복·조합 관련 도구 모음이에요. 'A, B, C 중 2개 뽑는 모든 경우' 같은 걸 직접 이중 `for` 로 짜지 않고 한 줄로 구해요.

파이썬 코드

```
import itertools

# 조합: 순서 상관 없이 2개 뽑기
combos = list(itertools.combinations(["A", "B", "C"], 2))
print(combos)

# 순열: 순서 구분해서 2개 뽑기
perms = list(itertools.permutations(["A", "B"], 2))
print(perms)
```

실행 결과

```
[('A', 'B'), ('A', 'C'), ('B', 'C')]
[('A', 'B'), ('B', 'A')]
```

조합(combinations)은 순서를 안 따져요 — (A,B)와 (B,A)를 같은 걸로 봐요. **순열(permutations)**은 순서를 따져요 — (A,B)와 (B,A)를 다르게 봐요. 복권 번호 조합, 팀 짜기 같은 문제에 바로 쓸 수 있죠.

파이썬 코드

```
import itertools

# count: 무한히 1, 2, 3, ... 을 내주는 카운터
counter = itertools.count(1)
print([next(counter) for _ in range(3)])
```

실행 결과

```
[1, 2, 3]
```

`itertools.count(1)` 는 1부터 무한히 세는 카운터예요(4장의 무한 제너레이터와 같은 원리). 필요한 만큼만 `next()` 로 꺼내 쓰면 돼요.

8.5 functools.reduce — 차곡차곡 하나로 접기

`reduce` 는 리스트의 값을 둘씩 묶어 가며 하나로 줄여요. 종이를 반씩 계속 접어 한 장으로 만드는 느낌이에요. 전체 합을 구해 볼게요(사실 합은 `sum()` 이 더 쉽지만, 원리 이해용이에요).

파이썬 코드

```
from functools import reduce

total = reduce(lambda a, b: a + b, [1, 2, 3, 4, 5])
print(total)
```

실행 결과

```
15
```

`reduce` 는 (((1+2)+3)+4)+5 처럼 왼쪽부터 둘씩 계속 합쳐 15를 만들어요. 곱하기(`a * b`)로 바꾸면 팩토리얼 같은 것도 만들 수 있죠. '여러 값을 한 값으로 뭉칠 때' 떠올리면 되는 도구예요.

8.6 pathlib — 경로를 객체처럼

파일 경로를 문자열로 자르고 붙이던 시절은 갔어요. `pathlib.Path` 는 경로를 똑똑한 객체로 다뤄, 파일명·확장자·존재 여부를 `점(.)`으로 깔끔하게 꺼내요.

파이썬 코드

```
from pathlib import Path

p = Path("/Users/choejunhyeon/python_course2/_verify/v08.py")
print(p.name)           # 파일 이름
print(p.suffix)         # 확장자
print(p.parent.name)    # 상위 폴더 이름
print(p.exists())       # 실제로 존재하는가
```

실행 결과

```
v08.py
.py
_verify
True
```

문자열을 `split("/")` 로 자르던 것에 비하면 훨씬 읽기 좋고 실수도 적어요. 폴더를 합칠 땐 `Path("폴더") / "파일.txt"` 처럼 슬래시로 이어 붙이기도 돼요. 요즘 파이썬 코드는 경로를 거의 `pathlib` 로 다뤄요.

8.7 json — 데이터를 문자열로 주고받기

웹 API, 설정 파일 등에서 데이터를 주고받는 표준 형식이 **JSON**이에요. 파이썬의 딕셔너리·리스트를 JSON 문자열로 바꾸거나 (`dumps`), 반대로 되돌려요 (`loads`).

파이썬 코드

```
import json

data = {"name": "지우", "age": 16, "hobbies": ["코딩", "독서"]}
json_str = json.dumps(data, ensure_ascii=False) # 딕셔너리 → 문자열
print(json_str)

parsed = json.loads(json_str) # 문자열 → 딕셔너리
print(parsed["name"])
print(parsed["hobbies"])
```

실행 결과

```
{"name": "지우", "age": 16, "hobbies": ["코딩", "독서"]}
지우
['코딩', '독서']
```



`ensure_ascii=False` 를 꼭 챙기세요. 이걸 빼면 한글이 `\uc9c0\uc6b0` 같은 암호로 저장돼요. `False` 로 두면 한글이 그대로 보여서 사람이 읽기 좋아요. 파일로 저장할 땐 `json.dump`, 읽을 땐 `json.load` (s 없는 버전)를 `with open` 과 함께 써요.

연습문제

1. 문자열 "apple banana apple cherry banana apple" 을 공백으로 나눠 Counter 로 세고, 가장 많은 단어를 most_common(1) 로 찾아 보세요.
2. deque 로 최근 기록 3개만 유지하는 코드를 만들어 보세요. (힌트: deque(maxlen=3) 에 계속 append)
3. 딕셔너리 {"city": "서울", "temp": 28} 을 json.dumps 로 ensure_ascii=False 옵션과 함께 문자열로 만들어 보세요.

해답 · 힌트

① Counter(text.split()).most_common(1) → [("apple", 3)] . ② q = deque(maxlen=3) 이후 넘치면 오래된 게 자동으로 밀려나요. ③ json.dumps(d, ensure_ascii=False) .

타입 힌트와 좋은 코드

파이썬은 변수 타입을 미리 안 정해도 되는 언어죠. 편하지만, 코드가 커지면 '이 함수는 대체 뭘 받아 뭘 돌려 주지?' 하고 헛갈려요. **타입 힌트**는 코드에 '이건 정수예요, 저건 문자열이에요' 라고 **메모**를 달아 두는 거예요. 여기에 **docstring·PEP8** 같은 습관까지 더하면, 몇 달 뒤의 나와 동료가 코드를 훨씬 편하게 읽어요. '읽기 좋은 코드'가 진짜 실력이에요.

이 장에서 배울 것

- 타입 어노테이션 기본 문법
- `typing` — List·Dict·Optional
- 왜 쓰는가 (강제가 아니라 안내)
- docstring 과 PEP8 스타일 맛보기

9.1 타입 어노테이션 — 코드에 메모 달기

함수 매개변수와 반환값에 `: 타입`, `-> 타입` 을 붙여 '무엇을 받고 무엇을 주는지' 표시해요. 실행 결과는 없어도 똑같지만, 읽는 사람에게 엄청난 정보를 줘요.

파이썬 코드

```
def greet(name: str) -> str:
    return f"안녕, {name}님!"

def add(a: int, b: int) -> int:
    return a + b

print(greet("지우"))
print(add(3, 5))
```

실행 결과

```
안녕, 지우님!
8
```

`name: str` 은 'name 은 문자열이에요', `-> str` 은 '이 함수는 문자열을 돌려줘요' 라는 뜻이에요. 함수 이름만 봐도 사용법이 그려지죠? 변수에도 `age: int = 16` 처럼 붙일 수 있어요.

9.2 typing — 리스트·딕셔너리 속까지 표현

'리스트'라고만 하면 안에 뭐가 든 리스트인지 모르죠. `typing` 모듈을 쓰면 `List[int]` (정수 리스트)처럼 속 내용까지 표현할 수 있어요.

파이썬 코드

```
from typing import List, Dict, Optional

def total(scores: List[int]) -> int:
    return sum(scores)

def get_price(menu: Dict[str, int], item: str) -> int:
    return menu[item]

print(total([90, 85, 100]))
print(get_price({"사과": 1000, "배": 2000}, "배"))
```

실행 결과

```
275
2000
```

`List[int]` 는 '정수들의 리스트', `Dict[str, int]` 는 '문자열 키에 정수 값인 딕셔너리'예요. 함수 서명만 봐도 데이터 구조가 한눈에 들어오죠. '없을 수도 있는 값'은 `Optional` 로 표현해요.

파이썬 코드

```
from typing import Optional

def find_user(user_id: int) -> Optional[str]:
    users = {1: "지우", 2: "민수"}
    if user_id in users:
        return users[user_id]
    return None      # 못 찾으면 None

print(find_user(1))
print(find_user(99))
```

실행 결과

```
지우
None
```

`Optional[str]` 는 '문자열이거나, 없으면 `None`'이라는 뜻이에요. '이 함수는 못 찾으면 `None` 을 줄 수 있으니 조심하라'고 미리 알려 주는 거죠. 덕분에 쓰는 쪽에서 `None` 검사를 잊지 않게 돼요.



파이썬 3.9+에선 `list[int]`, `dict[str, int]` 처럼 소문자 기본 타입으로도 쓸 수 있어요. 다만 `Optional` 같은 건 여전히 `typing` 에서 가져와요. 이 책은 어디서나 통하는 `typing` 방식으로 소개했어요.

9.3 타입 힌트는 '강제'가 아니라 '안내'

중요한 오해 하나를 짚고 갈게요. 타입 힌트는 **강제가 아니에요**. `int` 라고 적어 놓고 문자열을 넣어도 파이썬은 그냥 실행해요. 말 그대로 '메모/안내'일 뿐이죠.

파이썬 코드

```
def double(x: int) -> int:
    return x * 2

print(double("안녕"))  # int 라고 했지만 문자열도 그냥 돌아감
```

실행 결과

안녕안녕

`"안녕" * 2` 가 실행돼 "안녕안녕"이 나왔어요. 타입 힌트가 막아 주진 않죠. 그럼 왜 쓰냐고요? ① 사람이 읽기 좋고, ② 에디터 (VS Code 등)가 자동완성·오류 경고를 똑똑하게 해 주고, ③ `mypy` 같은 도구로 미리 검사할 수 있기 때문이에요. 실행을 바꾸진 않지만 개발을 훨씬 안전하고 편하게 만들어요.



타입 힌트를 쓰는 이유

- 코드가 문서가 돼요 (주석 없이도 사용법이 보임)
- 에디터의 자동완성·경고가 정확해져요
- 협업·유지보수 때 실수를 크게 줄여 줘요

9.4 docstring — 함수에 설명서 붙이기

함수 첫 줄에 삼중 따옴표로 쓰는 설명을 **docstring**이라고 해요. 이건 주석과 달리 `함수.__doc__` 로 꺼내 볼 수 있고, `help()` 에도 나와요. '이 함수가 뭘 하고, 뭘 받고, 뭘 주는지'를 적어 두면 최고의 설명서가 돼요.

파이썬 코드

```
def area(width: float, height: float) -> float:
    """직사각형의 넓이를 계산한다.

    Args:
        width: 가로 길이
        height: 세로 길이

    Returns:
        넓이(가로 x 세로)

    """
    return width * height

print(area(3, 4))
print(area.__doc__)
```

실행 결과

```
12
직사각형의 넓이를 계산한다.

    Args:
        width: 가로 길이
        height: 세로 길이

    Returns:
        넓이(가로 x 세로)
```

타입 힌트가 '무슨 타입인지'를, docstring 이 '무슨 일을 하는지'를 알려 줘요. 둘을 함께 쓰면 함수 하나가 스스로를 설명하는 완결된 문서가 되죠.

9.5 PEP8 — 파이썬의 스타일 약속

PEP8은 '파이썬 코드를 이렇게 쓰자'는 공식 스타일 가이드예요. 모두가 같은 스타일로 쓰면 서로의 코드를 읽기 훨씬 편하니까요. 핵심만 추리면 이래요.

항목	권장	비고
들여쓰기	공백 4칸	탭 대신 스페이스
변수·함수 이름	<code>snake_case</code>	소문자+밑줄
클래스 이름	<code>CamelCase</code>	단어마다 대문자
상수	<code>MAX_SIZE</code>	전부 대문자
연산자 주변	<code>a + b</code>	양옆 한 칸 띄우기
한 줄 길이	너무 길지 않게	보통 79~99자 권장



스타일을 손으로 다 지킬 필요는 없어요. `black` (자동 정렬), `flake8` (스타일 검사) 같은 도구가 자동으로 맞춰 줘요. '이런 약속이 있다'는 것만 알아 두고, 도구에게 맡기면 편해요. 지금은 일관성만 신경 쓰세요 — 한 프로젝트 안에서 스타일이 통일되면 충분해요.

연습문제

1. 두 정수를 받아 더 큰 값을 돌려주는 함수에 타입 힌트를 붙여 보세요. (`def bigger(a: int, b: int) -> int:`)
2. 문자열 리스트를 받아 전부 대문자로 바꾼 리스트를 돌려주는 함수에 `List[str]` 타입 힌트를 붙여 보세요.
3. 위 함수에 docstring 을 달고 `함수.__doc__` 로 출력해 보세요.

해답 · 힌트

① `return a if a > b else b` (또는 `max(a, b)`). ② `def upper_all(words: List[str]) -> List[str]:`
`return [w.upper() for w in words]`. ③ 함수 첫 줄에 `"""..."""` 를 넣으면 돼요.

파이썬답게: 자주 쓰는 패턴

같은 일을 해도 '파이썬다운(Pythonic)' 코드가 따로 있어요. 짧고, 읽기 좋고, 실수가 적은 관용 표현들이죠. 이 장은 그런 **실전 패턴 모음**이에요. 하나 하나는 작지만, 이걸 손에 익히면 코드가 눈에 띄게 깔끔해지고 '이제 좀 파이썬 쓸 줄 아네' 소리를 듣게 돼요.

이 장에서 배울 것

- 언패킹 — 여러 값 한 번에 풀기
- `enumerate` · `zip`
- `with` — 컨텍스트 매니저
- `*args` · `**kwargs` 심화와 예외 설계

10.1 언패킹 — 여러 값 한 번에 풀기

리스트나 튜플의 값을 변수 여러 개에 한 번에 나눠 담는 걸 **언패킹**이라고 해요. 특히 두 변수 값을 **맞바꾸는** 코드가 파이썬에선 임시 변수 없이 한 줄로 돼요.

● ● ● 파이썬 코드

```
a, b, c = [1, 2, 3]      # 리스트를 셋으로 나눠 담기
print(a, b, c)

x, y = 10, 20
x, y = y, x              # 값 맞바꾸기(임시 변수 없이!)
print(x, y)

first, *rest = [1, 2, 3, 4, 5]  # 첫 값 + 나머지 몰아 받기
print(first, rest)
```

실행 결과

```
1 2 3
20 10
1 [2, 3, 4, 5]
```

`x, y = y, x` 로 값을 맞바꾸는 건 파이썬의 상징적인 문법이에요. `*rest` 처럼 별표를 붙이면 나머지를 몽땅 리스트로 받아요. '첫 줄과 나머지 줄' 같은 걸 나눌 때 유용하죠.

10.2 enumerate — 번호와 값을 함께

반복하면서 순번(인덱스)도 필요할 때, `range(len(...))` 대신 `enumerate` 를 쓰는 게 파이썬다운 방법이에요.

파이썬 코드

```
fruits = ["사과", "배", "포도"]

for i, fruit in enumerate(fruits):
    print(i, fruit)

for i, fruit in enumerate(fruits, start=1): # 1번부터
    print(f"{i}번: {fruit}")
```

실행 결과

```
0 사과
1 배
2 포도
1번: 사과
2번: 배
3번: 포도
```

`enumerate` 는 (번호, 값) 짝을 만들어 줘요. `start=1` 을 주면 번호를 1부터 시작하죠. `for i in range(len(fruits)):` `fruits[i]` 같은 번거로운 코드는 이제 안녕이에요.

10.3 zip — 여러 리스트를 나란히

`zip` 은 여러 리스트를 같은 위치끼리 짝지어 줘요. 지퍼(zipper)가 양쪽 이를 하나씩 맞물리는 걸 떠올리면 돼요. 이름과 나이를 짝지어 볼게요.

파이썬 코드

```
names = ["지우", "민수", "하나"]
ages = [16, 17, 15]

for name, age in zip(names, ages):
    print(f"{name}: {age}세")

paired = dict(zip(names, ages)) # 두 리스트로 딕셔너리 만들기
print(paired)
```

실행 결과

```
지우: 16세
민수: 17세
하나: 15세
{'지우': 16, '민수': 17, '하나': 15}
```

`zip` 으로 두 리스트를 나란히 돌리고, `dict(zip(...))` 로 키-값 딕셔너리까지 한 방에 만들었어요(3장 컴프리헨션에서도 봤죠). 관련 있는 데이터가 여러 리스트로 흩어져 있을 때 `zip` 이 딱이에요.

10.4 with — 뒷정리를 자동으로 (컨텍스트 매니저)

1권에서 파일 열 때 `with open(...)` 을 썼죠. 그 `with` 의 정체가 **컨텍스트 매니저**예요. '다 쓰고 나면 알아서 닫아 주는' 안전장치죠. 블록을 벗어나는 순간, 정상 종료든 오류든 **무조건** 뒷정리(파일 닫기)를 해 줘요.

파이썬 코드

```
path = "memo.txt"

with open(path, "w", encoding="utf-8") as f:
    f.write("안녕하세요\n두번째 줄")
# 이 지점에서 파일이 자동으로 닫힘

with open(path, "r", encoding="utf-8") as f:
    content = f.read()
print(content)
```

실행 결과

```
안녕하세요
두번째 줄
```

`with` 블록을 벗어나면 파일이 **자동으로** 닫혀요. `f.close()` 를 직접 부를 필요가 없죠. 깜빡 잊고 안 닫는 실수를 원천 차단해 줘요. 파일뿐 아니라 데이터베이스 연결, 잠금(lock) 등 '열었으면 닫아야 하는' 자원에 두루 쓰여요.



with 를 쓰는 이유

오류가 나서 중간에 튕겨도 **뒷정리는 반드시 실행**돼요. '열고-쓰고-닫고'에서 **닫기를 잊는 실수**를 없애 주는 게 핵심이에요. 자원을 다룰 땐 `with` 를 습관처럼 쓰세요.

10.5 *args / **kwargs 심화

1권에서 맛본 `*args` (값 묶음)와 `**kwargs` (이름-값 묶음)를 제대로 써 볼게요. `**kwargs` 는 **이름 붙은 인자들**을 딕셔너리로 받아요.

파이썬 코드

```
def make_profile(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
make_profile(name="지우", age=16, city="서울")
```

실행 결과

```
name: 지우  
age: 16  
city: 서울
```

몇 개가 올지 모르는 이름 붙은 정보를 유연하게 받았어요. 반대로, 이미 리스트·딕셔너리에 든 값을 풀어서 함수에 넣는 것도 별표로 해요. 이것 '언패킹해서 넘기기'라고 해요.

파이썬 코드

```
def add3(a, b, c):  
    return a + b + c  
  
nums = [1, 2, 3]  
print(add3(*nums))           # 리스트를 풀어서 a, b, c 로  
  
info = {"a": 10, "b": 20, "c": 30}  
print(add3(**info))         # 딕셔너리를 풀어서 이름으로
```

실행 결과

```
6  
60
```

`add3(*nums)` 는 `add3(1, 2, 3)` 과 같아요. `add3(**info)` 는 `add3(a=10, b=20, c=30)` 과 같고요. '별 하나는 값 풀기, 별 둘은 이름-값 풀기'라는 규칙이 함수 정의와 함수 호출 양쪽에서 똑같이 쓰인다는 걸 알아 두세요.

10.6 예외를 활용한 설계

1권에서 `try/except` 로 오류를 막는 걸 배웠죠. 한 걸음 더 나가, 예외를 설계 도구로 쓸 수 있어요. '실패하면 안전한 기본값을 주는' 함수를 만들어 볼게요.

파이썬 코드

```
def safe_divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError:  
        return None      # 0으로 나누기 실패 → None 으로 안전하게  
  
print(safe_divide(10, 2))  
print(safe_divide(10, 0))
```

실행 결과

```
5.0  
None
```

0으로 나누면 프로그램이 죽는 대신 `None` 을 돌려줘 흐름을 이어가게 했어요. 9장의 `Optional` 과 짝지으면 '실패할 수 있는 함수'를 깔끔하게 설계할 수 있죠. '미리 다 검사하기보다, 일단 해 보고 실패하면 처리한다'는 게 파이썬다운 접근이에요(EAFP — Easier to Ask Forgiveness than Permission).

연습문제

1. `data = ["이름", "지우", "민수", "하나"]` 를 언패킹으로 `header` 와 나머지 `names` 로 나눠 담아 보세요. (힌트: `header, *names = data`)
2. 두 리스트 `subjects`, `scores` 를 `zip` 으로 짝지어 `enumerate` 로 번호까지 붙여 출력해 보세요.
3. `safe_int(s)` 함수를 만들어, 문자열이 숫자면 `int` 로 바꾸고 아니면 `None` 을 돌려주게 해 보세요. (힌트: `ValueError`)

해답 · 힌트

① `header, *names = data` → `names == ["지우", "민수", "하나"]`. ③ `try: return int(s) except ValueError: return None`.

동시성 맛보기

지금까지의 코드는 위에서 아래로 **한 줄씩 차례대로** 실행됐어요. 그런데 파일 10개를 인터넷에서 받는데, 하나 받고-기다리고-다음 받고... 하면 너무 느리죠. 여러 일을 **동시에** 처리하면 훨씬 빨라요. 이 장은 어려운 이론보다 '이런 게 있고, 이럴 때 쓴다'를 편하게 소개해요. 겁먹지 말고 개념만 잡아 가세요.

이 장에서 배울 것

- 동시성이 왜 필요한가
- `threading` vs `multiprocessing` 개념
- `concurrent.futures` 로 쉽게 병렬 처리
- 어떤 상황에 무엇을 쓸지

11.1 왜 동시성이 필요한가

일에는 두 종류가 있어요. **기다리는 일**(파일 다운로드, 웹 요청 — 대부분 '응답을 기다리는' 시간)과 **계산하는 일**(큰 수 계산, 이미지 처리 — CPU가 '열심히 일하는' 시간)이죠. 이 둘은 빠르게 만드는 방법이 달라요.

- **기다리는 일**이 많으면 → 여러 개를 동시에 기다리게 하면 빨라져요.
- **계산하는 일**이 많으면 → 여러 CPU에 나눠 계산하면 빨라져요.

비유하면, 라면 5개를 끓일 때 냄비 하나로 하나씩 끓이면 오래 걸리지만, 물 끓기를 **동시에** 기다리면 훨씬 빠른 것과 같아요.

11.2 threading vs multiprocessing

파이썬이 여러 일을 동시에 하는 방법은 크게 둘이에요. 개념만 표로 잡아 둘게요.

구분	threading (스레드)	multiprocessing (프로세스)
비유	한 사람이 여러 일을 번갈아	여러 사람이 각자 일함
잘 맞는 일	기다리는 일(다운로드·요청)	계산하는 일(무거운 연산)
메모리	공유 (가벼움)	각자 따로 (무겁지만 진짜 병렬)
핵심	대기 시간을 겹쳐 절약	CPU 여러 개를 진짜로 활용



파이썬에는 **GIL**이라는 특징이 있어서, **순수 계산**은 스레드를 여러 개 써도 잘 안 빨라져요. 그래서 '기다리는 일 = `threading`, 무거운 계산 = `multiprocessing`' 이라고 기억하면 대부분 맞아요. 지금은 이 구분만 알아도 충분해요.

11.3 threading — 여러 일을 번갈아

`threading` 으로 두 작업을 동시에 시작해 볼게요. 각 작업은 잠깐 기다리는 (`time.sleep`) 일을 흉내 내요.

파이썬 코드

```
import threading
import time

def worker(name):
    print(f"{name} 시작")
    time.sleep(0.1)    # 기다리는 일 흉내
    print(f"{name} 끝")

t1 = threading.Thread(target=worker, args=("작업1",))
t2 = threading.Thread(target=worker, args=("작업2",))
t1.start()           # 시작
t2.start()
t1.join()            # 끝날 때까지 기다림
t2.join()
print("모든 작업 완료")
```

실행 결과

```
작업1 시작
작업2 시작
작업1 끝
작업2 끝
모든 작업 완료
```

`start()` 로 두 작업을 거의 동시에 시작하고, `join()` 으로 '다 끝날 때까지 기다려' 했어요. 두 작업이 각각 0.1초씩 기다리지만, 겹쳐서 기다리니 전체는 0.1초 남짓이면 끝나요. 하나씩 했다면 0.2초가 걸렸겠죠.

⚠ 출력 순서는 매번 달라질 수 있어요

여러 스레드가 동시에 돌기 때문에, `작업1 시작` · `작업2 시작` 의 순서가 실행마다 바뀔 수 있어요. 이건 버그가 아니라 동시성의 자연스러운 성질이에요. '동시에 도는 것들은 순서를 보장하지 않는다'는 점을 기억하세요.

11.4 concurrent.futures — 가장 쉬운 병렬

스레드를 직접 만들고 `join` 하는 게 번거롭다면, `concurrent.futures` 가 정답이에요. `map` 처럼 쓰면 `알아서 병렬 처리` 하고 결과를 모아 줘요. 여러 값을 한꺼번에 제공해 볼게요.

파이썬 코드

```
from concurrent.futures import ThreadPoolExecutor

def square(x):
    return x * x

with ThreadPoolExecutor(max_workers=3) as executor:
    results = list(executor.map(square, [1, 2, 3, 4, 5]))
print(results)
```

실행 결과

```
[1, 4, 9, 16, 25]
```

`executor.map(함수, 리스트)` 는 5장의 `map` 과 사용법이 똑같아요. 다만 여러 일꾼(worker)이 나눠서 동시에 처리한다는 게 다르죠. `with` 로 감쌌으니(10장!) 끝나면 일꾼들이 알아서 정리돼요. 결과 순서는 입력 순서 그대로 유지돼 안심하고 쓸 수 있어요.

여러 사이트를 '다운로드'하는 상황(기다리는 일)을 흉내 내면 병렬의 이점이 잘 보여요.

파이썬 코드

```
from concurrent.futures import ThreadPoolExecutor
import time

def download(site):
    time.sleep(0.05)    # 네트워크 대기 흉내
    return f"{site} 완료"

sites = ["A", "B", "C", "D"]
with ThreadPoolExecutor(max_workers=4) as executor:
    results = list(executor.map(download, sites))
for r in results:
    print(r)
```

실행 결과

```
A 완료
B 완료
C 완료
D 완료
```

사이트 4개를 동시에 받으니, 하나씩($0.05 \times 4 = 0.2$ 초) 하는 것보다 훨씬 빨라요. 계산이 무거운 일이라면 `ThreadPoolExecutor` 를 `ProcessPoolExecutor` 로 바꾸기만 하면 여러 CPU로 병렬 처리돼요. 사용법은 거의 똑같아서 알아 끼우기 쉬워요.



실전 선택 가이드

- 인터넷 요청·파일 다운로드처럼 **기다리는 일** → `ThreadPoolExecutor`
- 무거운 **계산**(대량 연산·이미지 처리) → `ProcessPoolExecutor`
- 잘 모르겠으면 일단 `ThreadPoolExecutor` 로 시작해 보세요.



연습문제

1. `worker` 를 3개 만들어 `threading` 으로 동시에 실행하고 `join` 으로 모두 기다려 보세요.
2. `ThreadPoolExecutor` 와 `map` 으로 리스트 `[1, 2, 3, 4, 5]` 의 각 값을 세제곱해 보세요.
3. '기다리는 일'과 '계산하는 일' 중, 파이썬에서 `multiprocessing` 이 더 유리한 쪽은 무엇인지 말해 보세요.

해답 · 힌트

② `executor.map(lambda x: x ** 3, [1, 2, 3, 4, 5])` → `[1, 8, 27, 64, 125]` . ③ 계산하는 일(무거운 연산). GIL 때문에 순수 계산은 프로세스로 나뉘야 진짜 빨라져요.

실전 프로젝트

드디어 마지막 장이에요! 지금까지 배운 심화 개념들 — 클래스, 특수 메서드, 데코레이터, 제너레이터, 정규식, `Counter` — 을 **한데 모아** 진짜 프로그램 두 개를 만들어 봐요. 개념을 따로따로 아는 것과, 그걸 엮어 무언가를 만드는 건 완전히 다른 경험이에요. 한 줄씩 곱씹으며 따라오면, '아 이래서 배웠구나' 하고 퍼즐이 맞춰질 거예요.

이 장에서 배울 것

- 프로젝트 1 — 클래스로 설계한 도서 관리 시스템
- 프로젝트 2 — 로그 분석기 (데코레이터+제너레이터+정규식)
- 여러 개념을 엮어 실제 프로그램 만들기
- 완주! 스스로 코드를 설계하는 힘

12.1 프로젝트 1 — 도서 관리 시스템

도서관의 책을 등록하고, 대출·반납하는 시스템을 만들어요. 1·2장의 **클래스·상속·특수 메서드**가 총출동해요. 먼저 설계를 생각해 봐요. 두 개의 클래스가 필요하겠죠?

- **Book**(책): 제목·저자·대출 상태를 가진 하나의 책.
- **Library**(도서관): 책들을 모아 관리하고 대출·반납을 처리.

▶ Book 클래스 — 책 한 권 표현하기

책은 제목·저자와 '지금 대출 중인가'라는 상태를 가져요. `__str__` (2장!)로 예쁘게 출력되게 만들어요.

● ● ● Book 클래스

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_borrowed = False      # 처음엔 대출 안 됨

    def __str__(self):
        status = "대출중" if self.is_borrowed else "대출가능"
        return f"\u300e{self.title}\u300f - {self.author} [{status}]"
```

▶ Library 클래스 — 책들을 관리하기

도서관은 책 목록을 갖고, 등록·대출·반납·목록보기 기능을 메서드로 제공해요. 대출할 때 '이미 대출 중인지', '존재하는 책인지' 검사하는 **규칙**이 메서드 안에 담겨요.

```

class Library:
    def __init__(self, name):
        self.name = name
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        print(f"등록: {book.title}")

    def borrow(self, title):
        for book in self.books:
            if book.title == title:
                if book.is_borrowed:
                    print(f"'{title}'(은)는 이미 대출중입니다.")
                else:
                    book.is_borrowed = True
                    print(f"'{title}' 대출 완료!")
                return
        print(f"'{title}'(을)를 찾을 수 없습니다.")

    def return_book(self, title):
        for book in self.books:
            if book.title == title:
                book.is_borrowed = False
                print(f"'{title}' 반납 완료!")
                return
        print(f"'{title}'(을)를 찾을 수 없습니다.")

    def show_all(self):
        print(f"=== {self.name} 도서 목록 ===")
        for book in self.books:
            print(book)          # __str__ 가 자동 호출됨

```

▶ 실행해 보기

이제 두 클래스를 엮어 실제로 돌려 봐요. 책을 등록하고, 빌리고, 또 빌려 보고(실패), 없는 책도 빌려 보고(실패), 반납까지 해 봐요.

파이썬 코드

```
lib = Library("한빛 도서관")
lib.add_book(Book("파이썬 완전정복", "김코딩"))
lib.add_book(Book("자료구조 입문", "이알고"))
lib.add_book(Book("클린 코드", "박클린"))

lib.borrow("클린 코드")
lib.borrow("클린 코드")    # 이미 대출중
lib.borrow("없는책")       # 존재하지 않음
lib.return_book("클린 코드")
```

실행 결과

```
등록: 파이썬 완전정복
등록: 자료구조 입문
등록: 클린 코드
'클린 코드' 대출 완료!
'클린 코드' (은)는 이미 대출중입니다.
'없는책' (을)를 찾을 수 없습니다.
'클린 코드' 반납 완료!
```

보이나요? `print(book)` 한 줄이 `__str__` 을 자동으로 불러 예쁜 형태로 출력되고, 대출 규칙(중복없는 책 검사)이 `borrow` 안에 안전하게 들어 있어요. 데이터(책)와 규칙(대출 로직)이 클래스 안에 한 몸으로 담긴, 잘 설계된 프로그램이에요.



여기서 쓴 심화 개념

- 클래스·객체(1장): Book·Library 설계
- 특수 메서드(2장): `__str__` 로 예쁜 출력
- 캡슐화 사고: 대출 규칙을 메서드 안에 담아 안전하게

12.2 프로젝트 2 — 로그 분석기

서버 로그를 읽어 레벨별 개수와 에러를 낸 사용자를 집계하는 도구를 만들어요. 데코레이터·제너레이터·정규식·Counter 를 한번에 쓰는, 이 책의 하이라이트예요. 실무에서 로그 분석은 정말 자주 하는 일이라 바로 응용할 수 있어요.

분석할 샘플 로그예요(실제로는 파일에서 읽지만, 여기선 문자열로 둬게요).

● ● ● 샘플 로그 데이터

```
LOG_DATA = """2026-08-11 10:23:01 [INFO] user=hong login success
2026-08-11 10:23:05 [ERROR] user=kim login failed
2026-08-11 10:24:10 [INFO] user=lee page view
2026-08-11 10:25:33 [WARNING] slow response 3200ms
2026-08-11 10:26:01 [ERROR] user=park db timeout
2026-08-11 10:27:15 [INFO] user=hong logout
2026-08-11 10:28:40 [ERROR] user=kim login failed"""
```

▶ 제너레이터로 한 줄씩 꺼내기

로그가 아주 크다면 전부 메모리에 올리기 부담스럽죠. 4장의 **제너레이터**로 **한 줄씩** 꺼내 쓰면 메모리를 아낄 수 있어요.

● ● ● 제너레이터

```
def read_lines(data):
    """제너레이터: 한 줄씩 꺼내 준다."""
    for line in data.strip().split("\n"):
        yield line
```

▶ 데코레이터·정규식·Counter로 분석하기

이제 핵심 분석 함수예요. 6장의 **데코레이터**(`@timer`)로 감싸고, 각 줄에서 7장의 **정규식**으로 로그 레벨과 사용자를 뽑아, 8장의 **Counter**로 집계해요.

●●● 분석 함수

```
import re, functools, time
from collections import Counter

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        # 실행시간 측정(값은 매번 다름)
        return result
    return wrapper

@timer
def analyze(data):
    level_counter = Counter()
    error_users = []
    for line in read_lines(data):          # 제너레이터로 한 줄씩
        m = re.search(r"\\[(\\w+)\\]", line) # [INFO] 같은 레벨 추출
        if m:
            level_counter[m.group(1)] += 1
        if "[ERROR]" in line:
            um = re.search(r"user=(\\w+)", line) # user=kim 추출
            if um:
                error_users.append(um.group(1))
    return level_counter, error_users
```

▶ 실행 결과

●●● 파이썬 코드

```
levels, err_users = analyze(LOG_DATA)

print("=== 로그 레벨 집계 ===")
for level, cnt in levels.most_common():
    print(f"{level}: {cnt}건")

print("=== 에러 발생 사용자 ===")
print(Counter(err_users).most_common())
```


실행 결과

```
=== 로그 레벨 집계 ===  
INFO: 3건  
ERROR: 3건  
WARNING: 1건  
=== 에러 발생 사용자 ===  
[('kim', 2), ('park', 1)]
```

정규식 `\[(\w+)\]` 이 `[INFO]` 같은 대괄호 속 레벨을 잡고, `user=(\w+)` 가 사용자 이름을 뽑았어요. `Counter` 가 레벨 별·사용자별 개수를 세고, `most_common()` 이 많은 순으로 정렬했죠. 제너레이터로 줄을 흘려보내며 처리했으니 로그가 아무리 커도 메모리 걱정이 없고요. 짧은 코드 하나에 이 책의 심화 개념이 촘촘히 녹아 있어요.



여기서 쓴 심화 개념

- 제너레이터(4장): `read_lines` 로 한 줄씩 처리
- 데코레이터(6장): `@timer` 로 함수 감싸기
- 정규식(7장): 레벨·사용자 추출
- `Counter`(8장): 집계와 정렬

12.3 완주를 축하합니다!

여기까지 온 당신, 정말 대단해요. 1권에서 변수와 반복문을 처음 배우던 때를 떠올려 보세요. 이제는 **클래스로 프로그램을 설계** 하고, **데코레이터로 기능을 덧입히고**, **제너레이터로 메모리를 아끼고**, **정규식으로 데이터를 요리할** 줄 알아요. 더 이상 왕초보가 아니에요.

무엇을 배웠는지 짧게 되짚어 볼게요.

- 객체지향으로 데이터와 기능을 한 몸으로 묶는 설계 (1·2장)
- 컴프리헨션·제너레이터로 짧고 효율적인 코드 (3·4장)
- 일급 함수·클로저·데코레이터로 함수를 자유자재로 (5·6장)
- 정규식·표준 라이브러리로 실무 문제를 빠르게 (7·8장)
- 타입 힌트·파이썬다운 패턴으로 읽기 좋은 코드 (9·10장)
- 동시성의 개념과 `concurrent.futures` (11장)

이제 남은 건 **직접 만들어 보는 것**뿐이에요. 이 책의 프로젝트를 여러분의 아이디어로 바꿔 보세요. 도서 관리 시스템을 '나만의 가계부'로, 로그 분석기를 '채팅 기록 분석기'로 고쳐 보는 거죠. 막히면 이 책을 다시 펼치면 돼요. 코드는 읽기만 해선 늘지 않아요. 손으로 쳐 보고, 고쳐 보고, 망가뜨려 봐야 진짜 내 것이 돼요.

당신의 파이썬 여정에 이 책이 든든한 동반자였길 바라요. 이제 자신 있게 말해도 좋아요 — **"나, 파이썬 좀 하네."** 🐍